

Project 2

Metaheuristic Optimisation of TSP Using Genetic Algorithm with Repair Mechanisms

Mutiara Jassy Atikah (P169188)
Nur Ira Natasha Jafarizal (P168943)
Muhamad Ridzuan Mohd Yazid Goi (P172103)

Faculty of Information Science and Technology
Universiti Kebangsaan Malaysia

June 2026

Problem

Genetic Algorithm

Constraint Handling

Evaluation

Problem

Genetic Algorithm

Constraint Handling

Evaluation

Problem

Genetic Algorithm

Constraint Handling

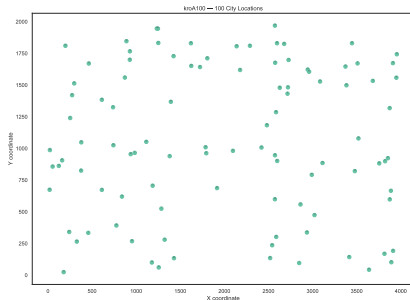
Evaluation

- ▶ 100 cities, symmetric Euclidean distances
- ▶ Known optimal tour length: **21,282**
- ▶ Search space: $\sim 4.7 \times 10^{155}$ distinct tours
- ▶ From TSPLIB (Reinelt, 1991)

Objective:

$$\min_{\pi} f(\pi) = \sum_{i=0}^{n-1} d(\pi_i, \pi_{(i+1) \bmod n})$$

where π is a permutation of $\{0, 1, \dots, 99\}$.



Permutation Encoding

Problem

Genetic Algorithm

Constraint Handling

Evaluation

- ▶ Chromosome = ordered tuple of city indices
- ▶ Example (8 cities): [3, 0, 7, 1, 5, 2, 6, 4]
- ▶ Tour: $3 \rightarrow 0 \rightarrow 7 \rightarrow 1 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 4 \rightarrow 3$
- ▶ Fitness = total closed tour length (lower is better)

Permutation Constraint

Every city must appear exactly once. A chromosome with duplicates or missing cities is **not** a valid tour, even though the fitness function still returns a number.

Selection

- ▶ Tournament ($k=3$) – primary
- ▶ Roulette wheel – comparison

Crossover

- ▶ Naive single-point – primary
- ▶ PMX – feasibility-preserving comparison

Mutation

- ▶ Swap mutation – sole operator
- ▶ Preserves permutation by construction

Other settings

- ▶ Elitism: top 2 carried over
- ▶ 500 generations, 30 seeds per config
- ▶ Pop size 200, $r_c=0.95$, $r_m=0.10$

Main Loop

```

1: Initialise population  $P$  of  $N$  random permutations
2: Evaluate fitness  $f(p)$  for all  $p \in P$ 
3: for  $g = 1$  to  $G_{\max}$  do
4:    $P' \leftarrow$  top- $e$  elites from  $P$ 
5:   while  $|P'| < N$  do
6:     Select parents  $p_1, p_2$  via tournament or roulette
7:     if  $\text{rand}() < r_c$  then
8:        $c \leftarrow \text{Crossover}(p_1, p_2)$ 
9:     else
10:       $c \leftarrow \text{copy}(p_1)$ 
11:    end if
12:     $c \leftarrow \text{Mutate}(c, r_m)$ 
13:    if repair enabled then
14:       $c \leftarrow \text{Repair}(c)$ 
15:    end if
16:    Append  $c$  to  $P'$ 
17:  end while
18:   $P \leftarrow P'$ ; evaluate fitness for all  $p \in P$ 
19: end for
20: return best individual in  $P$ 

```

▷ elitism

▷ swap

Lines 8 and 14 are the focus of this presentation: what goes wrong at crossover, and how repair fixes it.

Problem

Genetic Algorithm

Constraint Handling

Evaluation

Naive Crossover Breaks the Permutation

Problem

Genetic Algorithm

Constraint Handling

Evaluation

Naive Two-Point Crossover

	0	1	2	3	4	5	6	7	
Parent A	0	1	2	3	4	5	6	7	Segment: positions [2, 4]
Parent B	2	6	4	0	5	7	1	3	

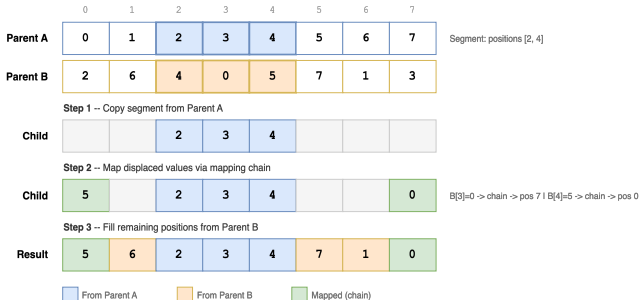
Blind splice -- copy A segment into B (no conflict resolution)

Result	2	6	2	3	4	7	1	3	
--------	---	---	---	---	---	---	---	---	--

Duplicates: {2, 3} -- Missing: {0, 5} -- **INVALID**

 From Parent A
 From Parent B
 Duplicate

Cities {2, 3} duplicated, cities {0, 5} missing. The child is **not** a valid tour – but the fitness function still returns a number, and it tends to be artificially low.

Partially-Mapped Crossover (PMX)

PMX copies a segment from Parent A, then uses a mapping to place the remaining values from Parent B without creating duplicates. Every city appears exactly once – no repair needed.

Our Approach: Repair After the Fact

Problem

Genetic Algorithm

Constraint Handling

Evaluation

Instead of switching to PMX, we keep naive crossover and **repair** every offspring. This way, repair has to do real work on every generation.

Phase 1 – Diagnosis: scan left to right, track *seen* set, flag duplicate positions, compute missing cities.

Phase 2 – Correction (random insertion): shuffle missing cities, place at duplicate positions. Does not favour any particular city arrangement, so population diversity is preserved.

Example

Input: (3, 1, 4, **1**, 5, 2, 6, **3**) Duplicates: {3, 7} Missing: {0, 7}

Random repair → (3, 1, 4, **7**, 5, 2, 6, **0**) ✓

Experiment Setup

Problem

Genetic Algorithm

Constraint Handling

Evaluation

Which strategy handles the permutation constraint best? We run the same GA three ways on kroA100:

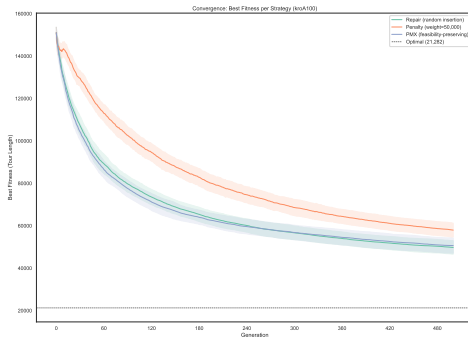
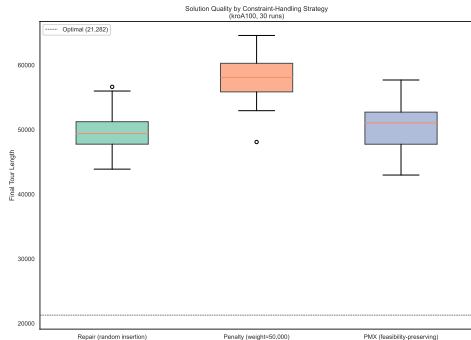
Shared parameters	pop=100, $r_c=0.8$, $r_m=0.2$, tournament, 500 gen
Seeds	30 independent runs per strategy

Repair Naive crossover + random-insertion repair. Every evaluated chromosome is a valid tour.

Penalty Naive crossover, no repair. Infeasible offspring penalised by $w \cdot v$ ($w=50,000$) added to fitness.

PMX Feasibility-preserving crossover. No repair needed.

Results



Strategy	Mean best	Std	Gap
Repair	49,778	3,187	133.9%
PMX	50,655	3,573	138.0%
Penalty	57,948	3,285	172.3%

Repair and PMX are almost the same; penalty is $\sim 16\%$ worse.

Problem

Genetic Algorithm

Constraint Handling

Evaluation

1. Naive crossover + repair performs almost as well as PMX on kroA100 (only $\sim 2\%$ difference)
2. Random insertion keeps more diversity than deterministic repair strategies
3. Penalty function is $\sim 16\%$ worse than both feasibility-based approaches
4. Fixing or preventing infeasibility works much better than just penalising it

Thank you